MODULE-1

BASICS OF C PROGRAMMING

INTRODUCTION TO COMPUTERS

The computer of each generation is smaller, faster and more powerful than preceding generation. There are five computer generations.

- **First Generation:** The vacuum tubes were used for computation. Magnetic drums were used for memory requirements. It consumed lot of space, power and generated lot of power. ENIAC (Electronic Numerical Integrator and Computer) used 18000 vacuum tubes, 1800 acquired sq. ft. room space and consumed 180KW of power. The machine level language (0s and 1s) was used. Punched cards were used for input and Paper for output. They were used for scientific work.

- **Second Generation:** The transistors was the most important component which replaced vacuum tubes. Magnetic cores were used for memory. It were more reliable than first generation computer. The assembly or symbolic language was used. The input and output mechanism remained same. The stored program concept was introduced which stores both data and program.

- **Third Generation:** The Integrated circuits(IC) was the most important component. The transistors, diodes, resistors, capacitors were integrated on a single chip. The high-level language was used like BASIC, C, C++ and JAVA. Memory capacity increased and magnetic hard disk was used for secondary generation. The third generation computers also had OS and computer could rum programs invoked by multi users.

- **Fourth Generation:** The Microprocessor was the most important component. With the help of LSI (Large Scale Integration) and VLSI (Very Large Scale Integration) the entire CPU is on a single chip. OS have moved from MSDOS to GUI (Graphical User Interface) like windows. The networking technology has also been improved. The size was reduced and the speed was increased.
- **Fifth Generation:** Artificial Intelligence and use of natural languages are the main features of this generations. These systems are expected to interact with users in natural language. Speech recognition and speech output should also be possible. Computers must be able to perform parallel processing. The quad-core and octa-core was also introduced. Neural networks and expert systems have been developed.

COMPUTER TYPES

Apart from being classified by generations, computers can also be categorized by their size. The size of a computer is often an indirect indication of its capabilities.

- **Supercomputers:** These are huge machines having most powerful and fastest processors. It uses multiple CPUs for parallel data processing. Speeds are measured in flops (floating point operations per second). The fastest operates at 34 petaflops. They are used for weather forecasting, analysis of geological data. They have enormous storage, uses more power and generate lot of heat. They are used by government agencies.

- **Mainframes:** These are multi-user machines that support many users using the feature of time sharing. It can run multiple programs even with a single CPU. The processor speed is measured in MIPS (Million instructions per second). It is used to handle data, applications related to organization and online transactions in banks, financial institutions and large corporations.

- **Minicomputers/Midrange computers:** It was introduced by DEC (Digital Equipment Corporation). They can serve hundreds of users and are small enough to partially occupy a room. They are used in smaller organizations or a department of a large one. They are not affordable to be used in home.

- **Microcomputers:** The microcomputer or PC is introduced by Apple and endorsed by IBM. This is a single-user machine powered by a single-chip microprocessor. They are very powerful machines having gigabytes of memory. They are both used in standalone mode and in a network. A microcomputer takes the form of desktop, notebook (laptop) or a netbook (smaller laptop). PCs today are powered by 3 types of OS – windows (7, 8 or 10), Mac OS X (Apple) and Linux. They are used for engineering and scientific applications and for software development.

- **Smartphones and Embedded Computers:** The smartphone is a general purpose computer i.e., capable of making phone calls. It has a powerful processor, with multiple cores, supports GBs of memory, and runs developed OS (Android or iOS). It can be operated with keyboard, touch or stylus.

  Embedded Computers or micro-controllers are very small circuits containing a CPU, non-volatile memory, input and output handling facilities. They are embedded into many machines that we use – cars, washing machines, cameras etc. The processor here runs a single unmodifiable program stored in memory.

BITS, BYTES AND WORDS

- Computer can understand only two states: 0 and 1.
- A digit can have only two states or values known as a binary digit, abbreviated as bit (b).
- The name nibble was coined to represent four bits.
- The name byte (B) was coined to represent eight bits.
- The byte is the standard unit of measurement of computer memory, data storage and transmission speed.
- The CPU handles memory data in larger units, called words and it is usually even multiple of bytes (two bytes, four bytes etc.).
- When referred to a computer it has 32-bit (4 bytes) machine i.e., size of word is 32 bits.

Unit	Equivalent to	Remarks
1 kilobyte (KB)	1024 bytes	Space used by 10 lines of text
1 megabyte (MB)	1024 kilobytes	Memory of the earliest PCs
1 gigabyte (GB)	1024 megabytes	Storage capacity of a CD-ROM
1 terabyte (TB)	1024 gigabytes	Capacity of today's hard disks
1 petabyte (PB)	1024 terabytes	Space used for rendering of film Avatar

COMPONENTS OF COMPUTER:

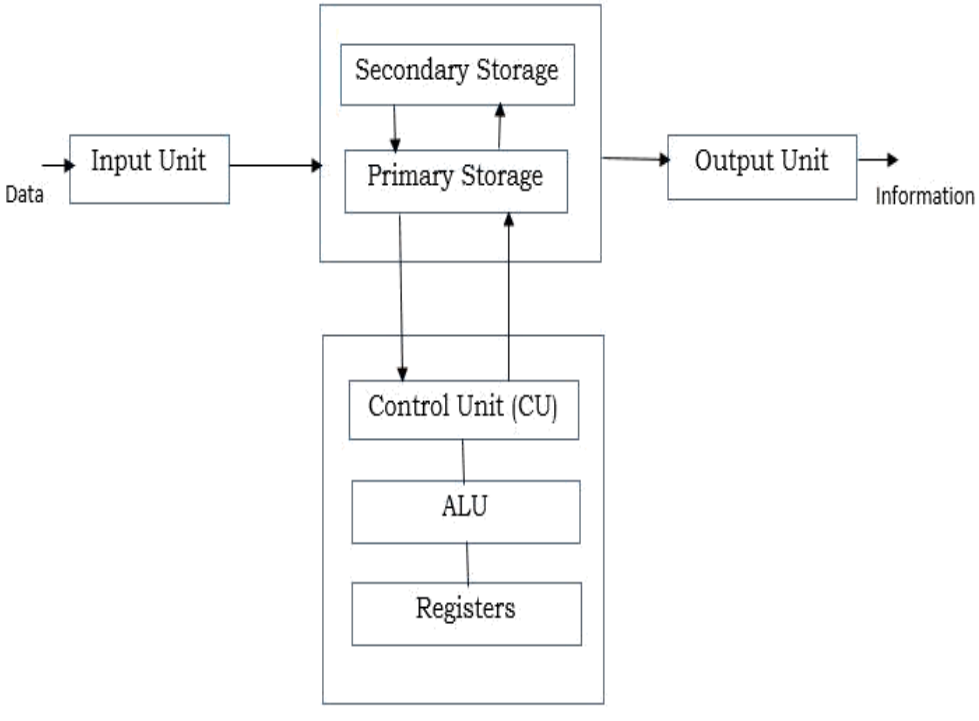


Figure 1: Functional Components of computer

The CPU comprises of following components:

- ALU (Arithmetic and Logic Unit)
- CU (Control Unit)
- Special-purpose registers
- A clock

The ALU is a "super calculator" carrying out all arithmetic tasks and Boolean operations (like AND, OR and NOT). The CU controls the way data is moved between the various components of computer. Both ALU and CU use the service of clock for synchronizing their Operations.

The CPU uses a few high-speed registers to store the current instruction and its data. One of the registers, program counter, stores the address of next instruction to be executed. All program instructions are executed using the fetch-decode-execute mechanism. An instruction is first fetched from memory, decoded and then executed by CPU. The CPU can execute one instruction in one clock pulse. Hence, faster the clock, faster is the execution. The CPUs today are rated in GHz (gigahertz).

PRIMARY MEMORY

Memory: Collection of chips on motherboard, where all computer processing and program instructions are stored.

Primary Memory: It is the main memory, which stores data and programs, which are currently needed by CPU. The size is less than the secondary memory and it is costly.

1. **RAM (Random Access Memory):** It is the main memory, where the user can write information into RAM and read information from it. It is accessible to users. The RAM is randomly accessible by user. It is a volatile memory, which means the data, or information is retained as long as power supply is ON. There are two types of RAM: SRAM and DRAM.
2.

SRAM (Static)	DRAM (Dynamic)
Stores information as long as power supply is on, reloads every 2ms.	Loses data in a very short time.
Multiple transistors are used to store 1 bit.	One transistor is used to store 1 bit.
It is expensive, faster, bigger.	It is inexpensive, slower, smaller.
It requires more power.	It requires less power.
It is not required to be refreshed.	It requires refresh.

2. **ROM (Read Only Memory):** It is a permanent memory that can be read but not written. It is a non volatile memory, which means the data or information is retained even power supply is not there. It contains a startup program BIOS (Basic Input Output System) which transfers control to OS.
   - **PROM (Programmable Read Only Memory):** It is programmed by the user. The data once written cannot be changed. Programmer burns the data into PROM. The data once

- **EPROM (Erasable Programmable Read Only Memory):** It can be rewritten (but only once) even though it has been previously burned. It is erased by exposing it to UV-rays.

- **EEPROM (Electrically Erasable Programmable Read Only Memory):** It can be erased and rewritten multiple times. The electric voltage is used to erase the data. The pen drive we use today is of EEPROM.

3. **Cache Memory:** It holds those portions of program that are frequently used by CPU. It acts as a buffer between CPU and RAM. The CPU first looks for the instructions in cache. It executes faster than RAM, expensive and limited in size. It has multiple levels:
   - **L1 (Level 1)** – smallest and fastest – 32 KB
   - **L2 (Level 2) –** present closer to CPU – 256 KB
   - **L3 (Level 3) –** shared by cores – 8 MB

4. **Registers:** The small number of ultra-fast registers integrated into the CPU represent the fastest memory of the computer. The CPU does all its work here. Each register has the length of the computer. The data is loaded into register before processing. Registers are numbered and a program instruction specifies these numbers. Ex: OR1R2R3 which means multiplication operation is performed on R1 and R2 is performed and stored in R3.

SECONDARY MEMORY

Secondary memory is not directly connected to CPU. It exists inside the machine and also externally. It is a non-volatile, offline and long-term storage memory. It is slower, cheaper than primary memory but the capacity is higher.

1. **Hard disk/ Hard drive/ Fixed disk:** It is the oldest secondary storage device. It has more capacity but, the cost is less comparatively. It is commonly present in laptop with 500GB and desktop with 1TB. It contains a spindle, which holds one or more platters made up of non-magnetic material. It has two surfaces which is coated with magnetic material. Each surface has serially numbered tracks and further broken into sectors or blocks. The disk runs with the speed of 5400 and 7200 rpm.

2. **Magnetic tape:** The magnetic tape is made up of plastic film with one side coated with magnetic material. It supports 1 TB or more, but 200 TB are also expected. The device is not fully portable though because, a separate tape drive is required. The data is accessed sequentially. This makes it unsuitable for backup.

3. **Optical Disks (CD-ROM, DVD ROM, Blu-ray Disk):** It is non-volatile read-only memory. The CD-ROM and DVD-ROM, can hold large volumes of data (700MB to 8.5 GB). The Blu-

ray disk has the capacity upto 50 GB. A laser beam in their drives controls the read and write operations. A laser beam is used to construct pits and lands by burning selected areas along its tracks.

CD-R, DVD-R: Data can be recorded only once.

CD-RW, DVD-RW: Data can be recorded multiple times.

4. **Flash Memory:** It doesn't have any moving parts, is based on the EEPROM. It is available in various forms- pen drive, magnetic card (SD Card), solid state disk (SSD). They are portable, need little power and quite reliable.

   The **memory stick or pen drive** is the most common type of flash memory used on the computer. It is a small, removable piece of circuit and it connects to the USB port of computer.

   The **solid state disk** is a bigger device meant to replace the traditional magnetic hard disk.

   The **magnetic card** is used mainly in cameras and the most popular is the micro-SD card.

5. **Floppy Diskette:** This is represented by a rectangular plastic case containing a thin magnetic disk. It was available in two sizes (5.25" and 3.5"), offering capacities of 1.2 MB and 1. MB. It is unsuitable for backup purpose.

✓ **Difference between Primary memory and Secondary memory**

Primary Memory	Secondary Memory
The size of the primary memory is small.	The size of the secondary memory is larger.
Stores the programs and data currently needed by the CPU.	It is used to store large amount of data and programs
Expensive.	Inexpensive.
Volatile in nature.	Non-volatile in nature.

INPUT DEVICES

Input devices are needed to interact with the OS to perform tasks. Ex: Keyboard, Mouse, Joystick, Stylus, Scanner etc.

➢ **Keyboard:** Every computer supports a keyboard – either a physical or touchscreen. The keyboard has QWERTY layout and contains large number of symbols. Each letter, numeral or symbol is known as a character, which represents smallest piece of information. Each character has a unique values called the ASCII (American Standard Code for Information Interchange) value.

➢ **Pointing Devices:** GUI (Graphical User Interfaces) like windows need a pointing device to control the movement of cursor on the screen. This is implemented as mouse in desktop and touchpad in laptop. The earliest form has a rotating ball and two buttons. The left button is used for selecting (by clicking once) and execute (by clicking twice). The right button is used to check and change attributes. The optical mouse uses infrared laser/LED, the wireless mouse uses radio frequency technology.

Scanner: A scanner is a device that creates a digital image of a document by optically scanning it. The flatbed scanner doesn't exceed more than A4 size. It is operated with by using special software. The document to be scanned is placed on a glass plate covered by lid. Modern scanners have **OCR (Optical Character Recognition)** facility which extracts text as a stream of characters i.e. converting image file to text file and **MICR (Magnetic Ink Character Recognition)** reads the codes using hand held barcode readers.

OUTPUT DEVICES

The information produced can be heard or seen with the help of output devices. Some of the output devices are monitor, speaker, printer, plotter etc.

➢ **Monitor:** The monitor is an integral part of computer which displays both text and graphics. The performance is measured in terms of image quality, resolution, energy consumption.

CRT (Cathode Ray Tube) Monitors: The CRT monitors have a rarefied tube containing three electron guns. The guns emit electrons create images. They usually have resolution of 640*840 pixels. They are large, heavy, energy efficient and produces lot of heat.

LCD (Liquid Crystal Display) Monitors: The image is formed by applying voltage on crystals. The backlight is provided by fluorescent light. They consume less power, generate less heat and have increased life span.

➢ **Impact Printers:** It produces the hardcopy of output. The impact printers are old, noisy. But, still dot-matrix printer is in usage.

Dot-Matrix Printer: The print head of the dot-matrix printer has either 9 or 24 pins. When the pins fire against the ribbon, an impression is created on the paper. The speed is 300 cps and doesn't produce high quality output. It can produce multiple copies.

Daisy-wheel Printer: It employs a wheel with separate characters distributed along its outer edge. The wheels can be changed to obtain different set of fonts.
Line Printer: For heavy printing, the line printer is used. It uses a print chain containing the characters. Hammers strike the paper to print and it rotates at the speed of 1200 lpm. It is also noisy and is of low-quality.

➢ **Non-Impact Printers:** They are fast, quiet and of high resolution. The most commonly used non-impact printers are laser and ink-jet printers. The thermal printer is not discussed here because it uses heat to print on high-sensitive paper.

Laser Printer: It works like a photocopier and uses toner i.e. black magnetic powder. The image is created in the form of dots and passed from drums on to the paper. It has built-in RAM which acts as buffer and RAM to store fonts. The resolution varies from 300 dpi to 1200 dpi and the speed is about 20 ppm.

Ink-jet Printer: These are affordable printers. It sprays tiny drops of ink at high pressure as it moves along the paper. The separate cartridges are available for different colors. The resolution is about 300 dpi, can print 1 to 6 pages/min.

- **Plotters:** The plotter can make drawings. It uses one or more automated pens. The commands are taken from special file called vector graphic files. Depending on type of plotter either paper or pen moves. It can handle large paper sizes and it is used for creative drawings like buildings and machines. They are slow and expensive.

Concept of Hardware(H/W) and Software(S/W):

The H/W of a computer consist of various electronic, electro-mechanical and electromagnetic devices along with interfacing buses connecting them.
The S/W of a computer consist set of instructions, commands and programs that make the computer to operate and fulfil our objectives. H/W is the body and S/W is the soul of a system.

SOFTWARE BASICS

- Software is a collection of code that drives a computer to perform related group of tasks.
- Programs in software use a language.
- The source code is created by programmer using programming languages like C, C++, Java, Python etc.
- The software is developed to operate on multiple platforms.

TYPES OF SOFTWARE:

2 Types i)System S/W and ii)Application S/W

- System S/W's: Operating S/M, Translator, Compilers, Assemblers, Linker and Loader.

Operating S/M: Acts as a interface between user and H/W. Ex of OS: Linux, Windows, Unix etc.
Functions of OS: CPU Management, Memory management, Device Management and File Management.
Translator: Translates a program Written in one language into another language.
Compilers: Translate a High level language program into machine level language(binary 0/1's).
Assembler: Translate a assembly language program(ADD,SUB,MULetc) into machine level language(binary 0/1's).
Linker: It converts object codes into executable codes this object codes are stored in Sec. memory And may be loaded into primary memory as and when required.
Loader: Loads program to be executed from sec memory to Pri. memory. Ex. Bootstrap Loader.

- **Application S/W's:** Are ones that are mean to perform a specific activity Ex: Word processing S/W, Image editing, Spread sheet, data base S/W and accounting S/W.

System Software	Application Software
System software is used for operating computer hardware.	Application software is used by user to perform specific task.
System soft wares are installed on the computer when operating system is installed.	Application softwares are installed according to user's requirements.
In general, the user does not interact with system software because it works in the background.	In general, the user interacts with application softwares.
System software can run independently.	Application software can't run independently.
It provides platform for running application Soft wares.	They can't run without the presence of system software.
Some examples of system softwares are compiler, assembler, debugger, driver, etc.	Some examples of application softwares are word processor, web browser, media player, etc.

Programming Language: The programming language can be classified into two categories:

- Low Level Language
- High Level Language

Low Level Language: The Low Level Language is classified into further two categories:

- **Machine Level Language:**
  Computers can understand only digital signals, which are in binary digits 1 and 0. So the instructions given to the computer can be only in binary code. The machine language consists of instructions that are written in binary codes.
  Advantages:
  - ❖ Since the computer can understand machine level language there is no need of any translator.
  - ❖ Program given to the computer in machine level language can be executed directly without conversion so, execution is very fast.

Disadvantages:

- ❖ Difficult to remember and write the machine instruction in binary code.
- ❖ It is very difficult to find and remove the errors.
- ❖ Program written in this language is machine dependent because each machine is having different architecture so, each will execute the given machine code in different way so we cannot use the program written in one computer on another computer.

- **Assembly Language:**
  The difficulties faced in machine level language were reduced to some extent by using assembly language. In assembly language instructions are given in English like word such as, MOV, ADD, SUB etc which are called as "mnemonic".
  Ex: MOV A, 4 : Store 4 in the register A of

CPU. MOV B, 5 : Store 5 in the register
B of CPU.
ADD A, B : Add the value in register A i.e. 4 and in B i.e. 5 and store the result in register A.

Advantages:

- It is bit easier to remember and write than machine level language because it is written in English.
- Program execution speed for assembly language is same as machine level language.
- It is relatively easy to find and remove the errors when compared to machine level language.

Disadvantages:

- Data is stored in registers so, program written in this language is machine dependent because there are different set of registers for different machines. Thus,we cannot use the program written in one computer on another computer.
- It is still difficult to remove errors and modify the program when compared to the High level language.

High Level Language: A High level language is one, which is written using symbols and word just like English language.

Advantages:

- These languages are much easier to write and remember and understand.
- These languages enable programmer to write a machine independent code, because computer will have translators to translate this code into machine level language. So, each language is having its own translator that will translate the same program on different machines according to the machine architecture into machine level language.
  Example: FORTRAN, COBOL, BASIC, Pascal, etc.

Disadvantages:

- The program execution will take more time than low level languages because first it has to be translated in machine level language.

Pseudocode

- Pseudocode is a type of structured English written in natural language, to describe set of rules for step-by-step processes, problem solving operations or other calculations.
- It is informal, high level description of a computer program intended for human reading rather than machine reading.
- The aim for writing pseudocode is to get idea quickly.
- There are no structure rules for writing pseudocode and it doesn't require program syntax.
- It is used in planning of computer program development, for sketching out the structure of the program before the actual coding takes place.

Advantages:

1. It can be written easily
2. It can be read and understood easily
3. Modification is easy
4. It implements structured concepts easily

Disadvantages:

1. It is not visual
2. There is no standardized style or format, so one pseudocode may vary from other for same problem statement

Examples:

Pseudocode: To compute addition of 2 no.	**Pseudocode:** To compute area of rectangle
Start Get the values of number1 and number2 Calculate addition=number1+number2 Print addition Stop	Start Get length and width Calculate area=length*breadth Print area End

HISTORY OF C

- C is a general purpose, procedural, structured computer programming language developed by **Dennis Ritchie** in the year 1972 at AT&T Bell Labs.
- C language was developed on UNIX and was invented to write UNIX system software.
- C is a successor of B language.
- There are different C standards: K&R C std, ANSI C, ISO C.

Characteristics of C:

- Middle-level language combining features of high-level and low-level languages
- C is easy to learn.
- C is a general purpose language.
- C is a structured and procedural language.
- It is portable.
- It can extend itself.

Examples of C:

- Operating system
- Language compilers
- Text editors
- Databases

Designing efficient C programs:

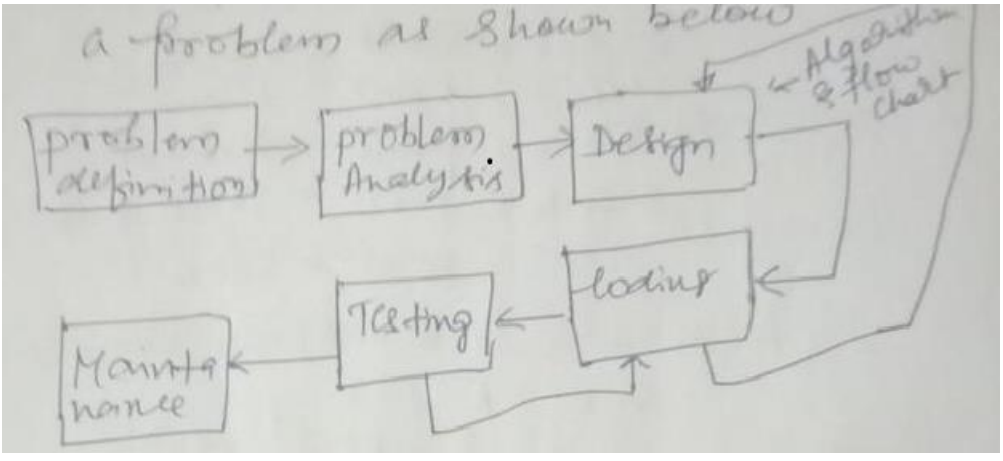
1. Program Planning and Design

Before writing code, proper planning is essential for program efficiency:

- Define clear program objectives and expected outcomes.
- Break down complex problems into smaller, manageable modules.
- Design appropriate algorithms for each module.
- Create flowcharts or pseudocode to visualize program logic.
- Identify potential bottlenecks and optimization opportunities early.

STEPS TO SOLVE A PROBLEM **OR** PROGRAM DEVELOPMENT LIFE CYCLE OF C.

There are 6steps as shown below.



Problem Definition: Programmer first must define the Physical problem or to be define the requirement of particular Solution.

Problem Analysis: When the problem is received, the programmers try to understand the problem at first. They determine whether the problem is a valid one and can have a solution or not. If the problem is valid, then they chalk out the various solutions that it may have. Then a single feasible solution is selected and the boundaries of problem are identified.

Design: In order to attain the selected solution, a step by step procedure is developed. The variables, Files, Functions etc are determined. The logical flow is determined in this phase. The tools used are flow chart, algorithm, Structured chart, decision table and pseudocode.

Coding: The algorithms developed in the previous phase is converted into C language code. The programmer should follow the syntax of C language while writing the code. The code is periodically compiled to identify the errors and remove them.

Testing: Once the code is ready program is executed to see the output. The output may be correct or incorrect if so there is a feedback to coding and design part until the design is correct**.**

Maintenance: Any changes made to the program after installation is called maintenance. Periodic maintenance is done to fulfil the changing requirement of the customer. A very important activity that needs to be done throughout each phase of program development is documentation.

ALGORITHMS AND FLOWCHARTS

Algorithms

An algorithm is a self-contained step-by-step set of operations to be performed. Algorithms perform calculation, data processing, and/or automated reasoning tasks.

The characteristics of a good algorithm are:

- Precision – the steps are precisely stated(defined).
- Uniqueness – results of each step are uniquely defined and only depend on the input and the result of the preceding steps.
- Finiteness – the algorithm stops after a finite number of instructions are executed.
- Input – the algorithm receives input.
- Output – the algorithm produces output.
- Generality – the algorithm applies to a set of inputs.

Advantages of algorithms

- Algorithm is a step-by-step rep. of a solution to a given problem, which is very easy to understand
- It has got a definite procedure.
- It is easy to first develop an algorithm & then convert it into a flowchart &then into a computer program.
- It is independent of programming language.
- It is easy to debug as every step has got its own logical sequence.

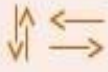
Disadvantages of algorithms

- it is time consuming & cumbersome as an algorithm is developed first which is converted into flowchart & then into a computer program.
- Understanding complex logic through algorithms would be difficult

Flowchart

A **flowchart** is a type of diagram that uses an algorithm, workflow or process, showing the steps as boxes of various kinds, and their order by connecting them with arrows. This diagrammatic representation illustrates a solution model to a given problem. Flowcharts are used in analyzing, designing, documenting or managing a process or program in various fields.

List of different symbols and their purpose

Picture	Shape	Name	Action Represented
	Oval	Terminal Symbol	Represents start and end of the Program
	Parallelogram	Input/Output	Indicates input and output
	Rectangle	Process	This represents processing of action. Example, mathematical operator
	Diamond	Decision	Since computer only answer the question yes/no, this is used to represent logical test for the program
	Hexagon	Initialization/ Preparation	This is used to prepare memory for repetition of an action
	Arrow Lines & Arrow Heads	Direction	This shows the flow of the program
	Circle	On page connector	This is used to show connector or part of program to another part.

Hexagon: For Looping.

Badge Symbol: Refer Notes: Inter Page Connector.

Advantages of Using FLOWCHARTS:

1. Communication: Flowcharts are better way of communicating the logic of a system to all concerned or involved.
2. Effective analysis: With the help of flowchart, problems can be analyzed in more effective way therefore reducing cost and wastage of time.
3. Proper documentation: Program flowcharts serve as a good program documentation, which is needed for various purposes, making things more efficient.
4. Efficient Coding: The flowcharts act as a guide or blueprint during the systems analysis and program development phase.
5. Proper Debugging: The flowchart helps in debugging process.

6. Efficient Program Maintenance: The maintenance of operating program becomes easy with the help of flowchart. It helps the programmer to put efforts more efficiently on that part

Disadvantages of Using FLOWCHARTS:

1. Complex logic: Sometimes, program logic is quite complicated. In that case, flowchart becomes complex and clumsy. This will become a pain for the user, resulting in a waste of time and money trying to solve the problem.

2. Alterations and Modifications: If alterations are required the flowchart may require re-drawing completely. This will usually waste valuable time.

3. Reproduction: As the flowchart symbols cannot be typed, reproduction of flowchart becomes a problem.

Example 1: Write an algorithm and Flow chart to find area and circumference of a circle.

```
  Step 1: Start

  Step 3: Read value of radius

  Step 4: computer area and circumference

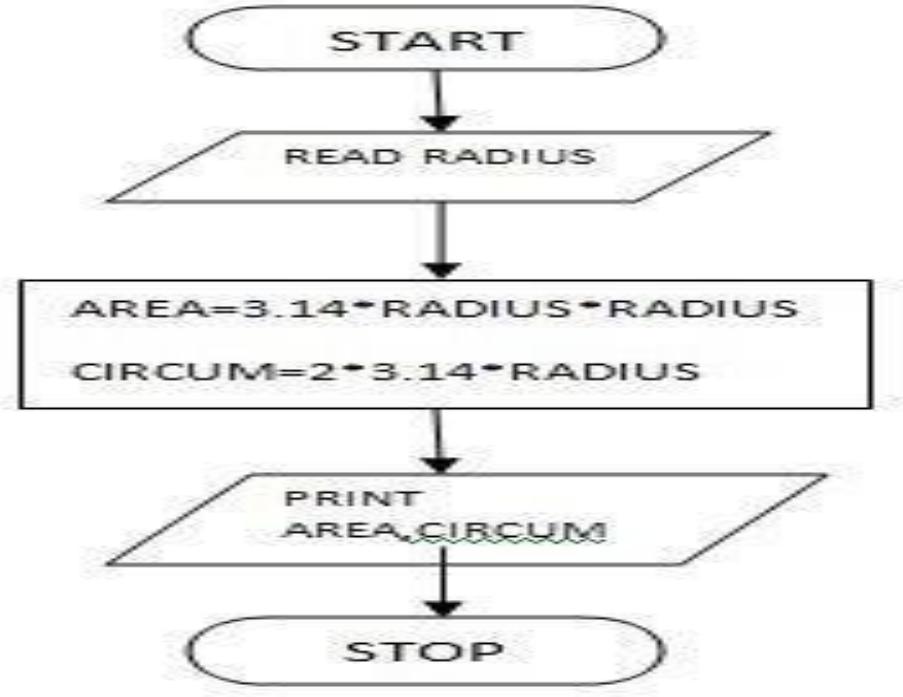
        Area ← 3.14*radius*radius

Circumference= 2 * 3.14 * radius

  Step 5: print area and circumference

  Step 6: Stop
```

Flow Chart:



Example 2: Write an algorithm to find the largest among two different numbers entered by user.

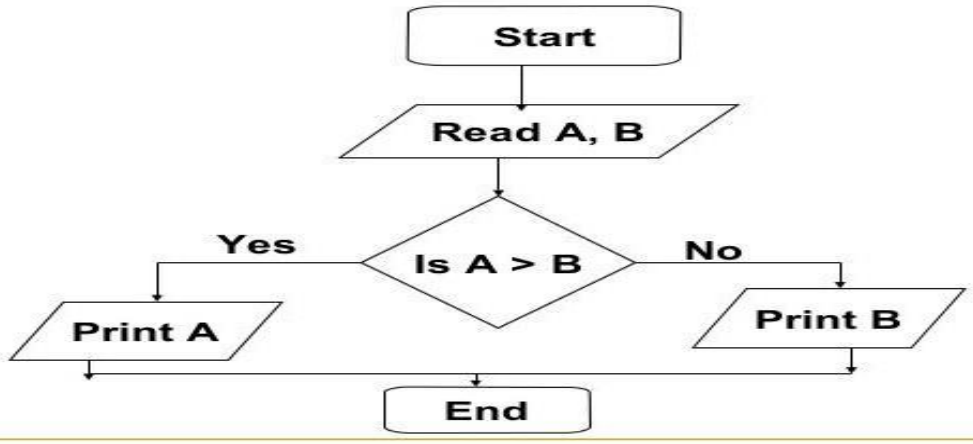
```
Step 1: Start
Step 2: Read two numbers a,b
Step 3: If a>b
              Display a is the largest number.

           Else

             Display b is the largest number.

Step 4: Stop
```

Flow Chart to find largest of two numbers:



Example 3: Write an algorithm to find whether the given number is even or odd.

```
Step 1: Start

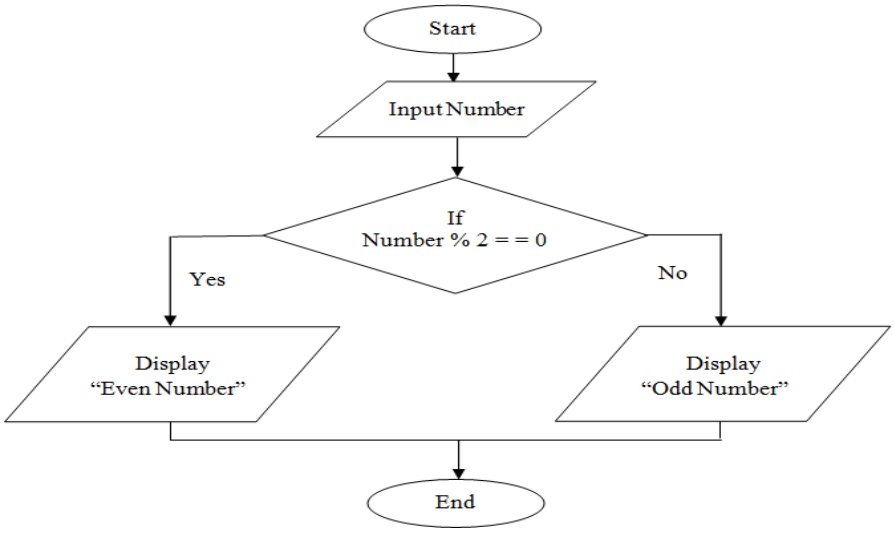
Step 2: Read N

Step 3: compute remainder

        Rem ← N % 2

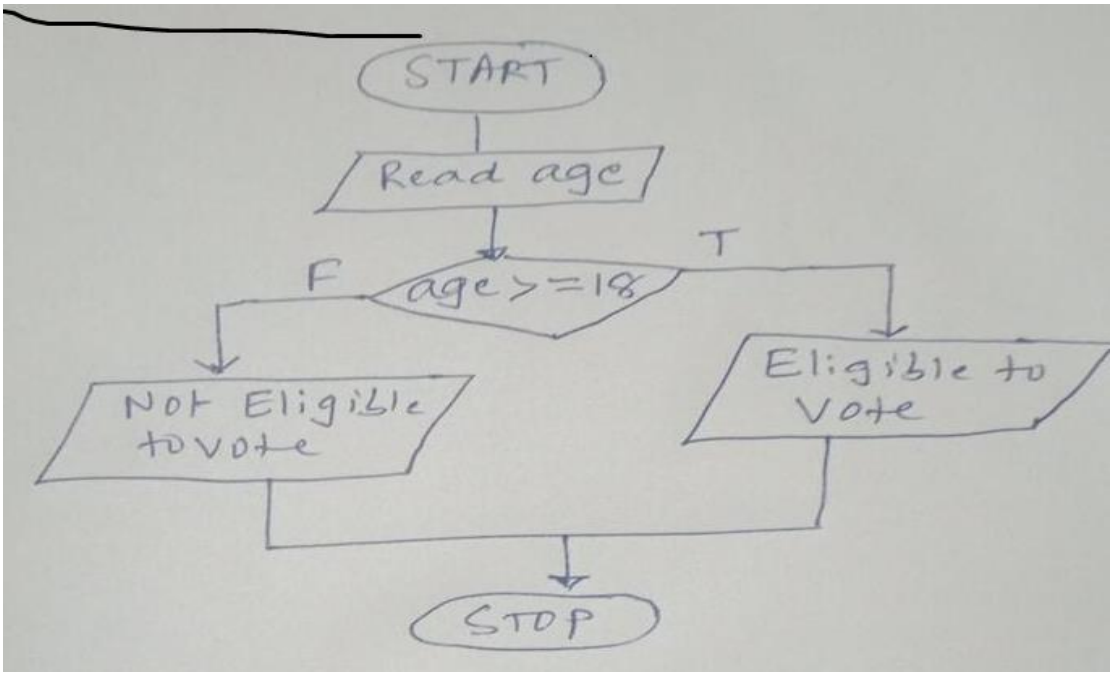
Step 4: if Rem==0 then

        Print “even”
        Else
        Print “odd”
Step 5: Stop
```



Example 4: Write an algorithm to find whether the person is eligible to vote.

```
Step 1: Start
Step 2: Read age of a person
Step 3: If age>=18
              Display person is eligible to vote.
           Else
             Display person is not eligible to vote.
Step 4: Stop
```



Example 5: Write an algorithm to find whether the given number is positive or negative.

```
Step 1:

Start Step

2: Read a

Step 3: If

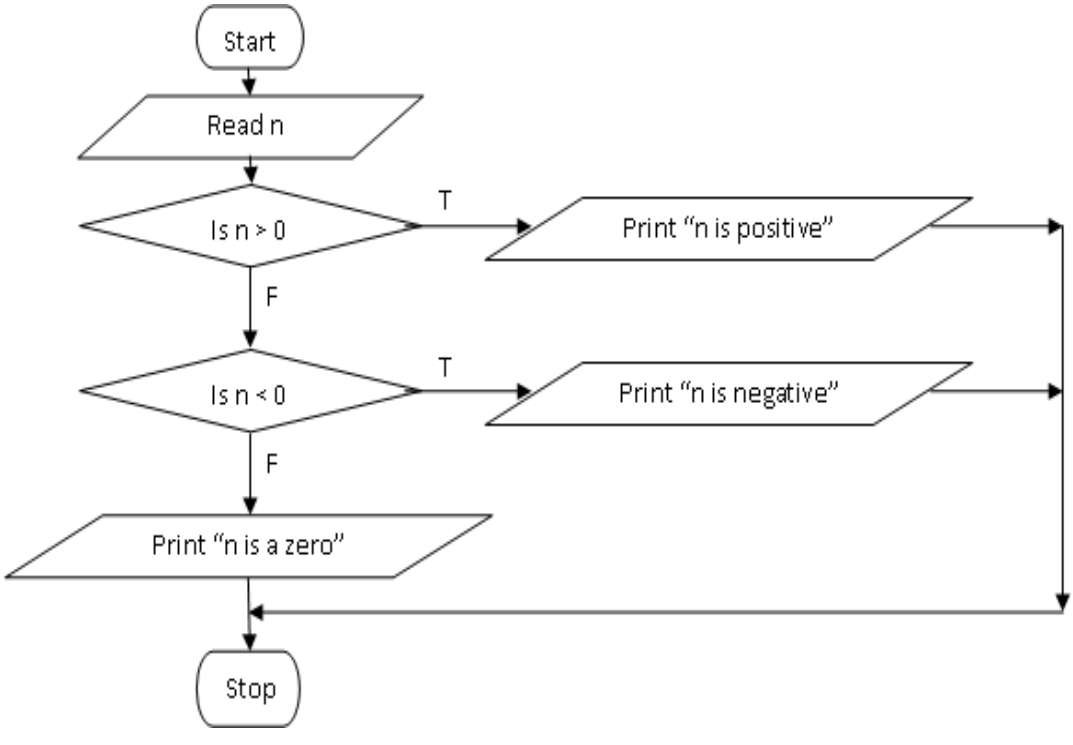
a>0

              Display a is positive.

           Else

             Display a is negative

Step 4: Stop
```



Basic structure of C Program

Documentation Section. -Optional
Link Section (#include section).-Mandatory
Definition section.- Optional
Global Declaration Section- Optional

main() Function-Mandatory
{
Local Declarations Statements
Execution Part
}

SubProgran section- Optional Function-1 Function-2 …….. …….. Function-n These are User defined functions.

C program consists of different sections. The structure of C program is as shown below.

Documentation Section: It consists of "comments" which increases the readability of program. The comments are not processed by the compiler. There are two types of comments:

a) Single line comment(Only one line) //Single line comment.

b) Multi line comment /*Multi line comment*/

Link Section: It provides instruction to the compiler to link functions from system library.

Definition Section: defines all symbolic constants.

Global Declaration section: there are some variables that are used in more than one function, such a variable are called global variable and are declared in the global declaration section.

main ( ) function section: Every C program must have one **main ( ) function section**. This section contains two parts declaration part and executable part

Declaration part declares all the variables used in the executable part

There is at least one statement in an executable part. These two part must appear at the beginning of the brace and ends at the closing brace. All statement in the declaration and executable part ends with semicolon.

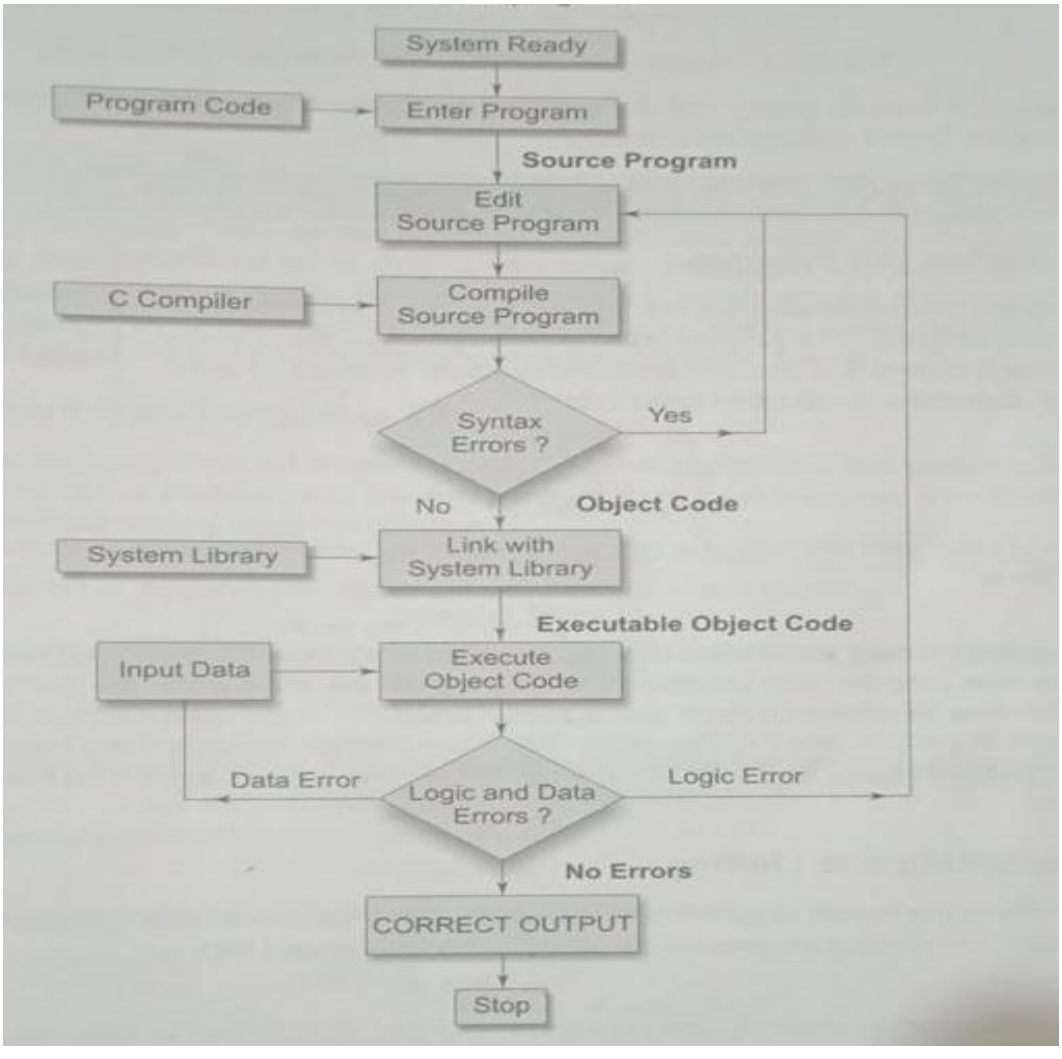
The sub program section contains all the user defined functions that are called in the main function

Here is a small program that displays a sentence "Welcome to C world " on the monitor screen:

```
/* C program to display welcome to C world */
#include<stdio.h>
void  main( )

{
        printf("Welcome to C World");
        }
```

COMPILING AND EXECUTING A C PROGRAM



Executing a C program involves series of steps.

1.Creating the program

2.Compiling the program

3.Linking the program with functions that are needed from the C library and

4.Executing the program. As shown in the above figure.
An operating system is a program that controls the entire operation of a computer system. OS is a Interface between hardware and user, handles execution of user programs.

Source code is the file containing the C source program. Compiler is a S/m program that translates the source program to an object code. The object code contains the equivalent set of machine instructions corresponding to the source program which is understand by the computer. Object code of the program will be linked with the object code of library functions by linker which is S/M software to produce the final executable code the generation of executable code is shown below.

FILES USED IN C PROGRAM:

The files used in a C program can be categorized into several types, all working together to create a final executable. This process generally involves a sequence of steps, from writing the source code to linking it with other necessary files.

Source Code Files

These are the files you write, containing the C language instructions. They typically have a **.c** extension. For example, a program to calculate the area of a circle might be saved as `area.c`.

Header Files

Header files, denoted by the **.h** extension, contain function prototypes, macro definitions, and variable declarations. They are included in source code files using the `#include` preprocessor directive. For instance, the `<stdio.h>` header file is included for standard input/output functions like `printf()` and `scanf()`. When you include a header file, its contents are effectively copied into your source code file before compilation begins.

Object Files

After the C preprocessor and compiler have processed the source code, they produce an **object file**. This file contains machine code for your program but is not yet a complete executable. It has a **.obj** extension on Windows and **.o** on Linux/macOS. This file holds compiled code but lacks the necessary external function definitions from libraries.

Library Files

Library files contain pre-compiled code for standard functions, such as those for mathematical operations or string manipulation. The **linker** uses these libraries to resolve the function calls in your object files. A key concept here is that the library code is not part of your source file; it's a separate collection of compiled functions that the linker connects to your program's object file. Examples include the standard C library (`libc`) which contains a vast collection of common functions.

Executable Files

The final product of the compilation and linking process is the **executable file**. This is a standalone file that can be run directly by the operating system. It typically has an **.exe** extension on Windows and no specific extension on Unix-like systems. This file contains all the necessary code, including your compiled source code and the linked library functions, to run your program.

File extension for C Files:

Source Code Files: XYZ.c

Header Files: .h

Object Files: .obj/.o

Executable Files: .exe

Character set of C:

The characters in C are grouped into the following **two** categories:

- **ALPHABETS**
  - Uppercase letters A-Z
  - Lowercase letters a-z
- **DIGITS:** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

- **SPECIAL CHARACTERS** Here are some special characters that can be used in C:

Character	Name	Character	Name	Character	Name
+	Plus sign	-	Minus sign (hyphen)	*	Asterick (multiplication)
/	Forward slash (Division)	%	Percentage sign (modulus)	\	Backward slash
<	Less than	>	Greater than	=	Equal to (Assignment)
_	Underscore	(	left parenthesis	)	right parenthesis
{	left flower brace	}	right flower brace	[	left bracket
]	right bracket	,	comma	.	dot operator (period)
‘	Apostrophe (Single Quotes)	“	Double Quotes	;	Semi Colon
:	Colon	?	Question mark	!	Exclamation mark
&	Ampersand	|	Vertical Bar	~	Tilde sign
^	Caret Sign (Power)	#	Hash		

Blank, newline, vertical tab, horizontal tab, carriage return are known as whitespace in C language. Following are some escape sequences used in C:

Meaning	Escape Sequence	Use
Backspace	\b	Moves the cursor to the previous position of the current line
Horizontal tab	\t	Moves the cursor to the next horizontal tab position
Vertical tab	\v	Moves the cursor to the next vertical tab position
New line	\n	Moves the cursor to the beginning of the next Line
Carriage return	\r	Moves the cursor to the beginning of the current line
Null	\0	Null Character used for termination of character string
Alert	\a	Produce and audible or visible alert

Backslash	\\	Present a character backslash

Note: Characters are always defined inside a single coat:

Ex: 'a' '+' 'A' '-'

C Tokens: C Tokens are simple elements to build a C program. C has 6 Different types of tokens.

1.Keywords
2.Identifiers
3.constants
4.Variables
5.Operators
6.Strings and special symbols.

Let us discuss each one of them in detail:

1.Keywords: Keyword or reserved word have predefined meanings and these meanings cannot be changed. Keyword serves as building blocks for program statements. All keywords are in lowercase letters. Below table shows all keywords in ANSI C.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
const	extern	return	union
char	float	short	unsigned
continue	for	signed	volatile
default	goto	sizeof	void
do	if	static	while

Identifiers: In C language identifiers are the names given to variables, constants, functions and user-define data. These identifiers are defined against a set of rules.

Rules for an Identifier

1. An Indetifier can only have alphanumeric characters(a-z , A-Z , 0-9) and underscore(_).
2. The first character of an identifier can only contain alphabet(a-z , A-Z) or underscore (_).
3. Identifiers are also case sensitive in C. For example *name* and *Name* are two different identifiers in C.
4. Keywords are not allowed to be used as Identifiers.
5. No special characters, such as semicolon, period, whitespaces, slash or comma are permitted to be used in or as Identifier.

Example of some valid identifiers: Value, a, net_ pay, data, Marks, Int (its not keyword here I is uppercase)

Example of some invalid identifiers:

- 5a (First character should be an alphabet of underscore)
- int (int is a keyword)
- rec# (special character is not allowed)
- avg number (space is not allowed in identifier name)

Constants: The constant is a data item which will not change during the execution of a program. The classification of constants in C is shown below.

Numeric constants:

- Integer constants
- Real constants

Character constants:

- Single character constants
- String constants.

Integer constants: Integer constants are whole numbers without any fractional part. No extra characters are allowed other than + and - sign. Integer constants can be written in three different number systems: Decimal, Octal and Hexadecimal

A **decimal** integer constant consists of any combination of digits taken from the set 0 to 9. If the decimal constant contains two or more digits, the first digit must not be 0. Only 0 is a decimal integer. The following are valid decimal integer constants.

Example: 0 1 1234 -786

The following decimal integer constants are written incorrectly for the reasons stated.

12,001	Illegal character (,).
103.0	Illegal decimal point (.).
10 20 30	Illegal character (blank space).
045	First digit cannot be zero.

An **octal** integer constant can consist any combination of digits taken from the set 0 through 7. However, the first digit must be 0, in order to identify the constant as an octal number. The following are valid octal integer constants.

Example: 01 0125 0555

A **hexadecimal** integer constant must begin with either 0x or 0X followed by any combination of digits taken from the set 0 through 9 and A through F (either upper-case or lower-case). The following are valid hexadecimal integer constants.

Example: 0X0 0x1 0XAB125 -0x555

Real constants or Floating Point Constant: A floating-point constant that contains either a decimal point or an exponent or both.

- A **Fractional** constant in a fractional form must have at least one digit each to the left and right of the decimal point. The left part of decimal point is called as mantissa and right part is called as exponent.

  Example: 5.8 8.9567 679.4678

- The **exponent** form is used to represent a floating point number if it is either too large of too small.
-

Example: $2.3*10^{67}$ can be written as 2.3e67 or 2.3E67
$9.77*10^{-77}$ can be written as 9.77e-77 or 9.77E-77

Character Constant: A character constant is a single character, enclosed in single quotation marks.

Example: 'A' 'B' '1'
Each character takes 1 byte in memory**.**

String Constant: String is "Sequence of one or more Characters". String Constant is written in Pair of Double Quotes. In C, String data type is not available. The compiler will add the end of string character or NULL character '\0' at the end of each string by default.
Each String Constant takes 4 byte in memory.

Example: "AMIT" is a string takes 5 bytes (4 bytes for AMIT and 1 byte for NULL character)

Note: 1 byte=8 binary bits 1Nibble=4 binary bits

Special constant called **define constants**: they are defined in the beginning of program along with pre-processor
directives like #include<stdio.h>
Examples: **#define** PI 3.147
#define RATE_INTEREST 10
Once we define such values, we can use names like PI, RATE_INTEREST throughout the C programs with the specified values. That is wherever PI is used it is substituted with 3.147 and RATE_INTEREST with 10.

Const is a keyword makes memory location called 'pi' to have a constant value 3.147 throughout program execution.

Variables: Variable is a data name that may be used to store data value.A variable may take different values at different times during execution.

- Rules to choose a variable name:
  - Must begin with a letter or underscore.
  - Case sensitive, Total not same as total.
  - Should not be a key word.
  - White space is not allowed.

Some valid variables: Value, sum1, mark and amount, average_ number etc.
Some invalid variables: Group one (white space not allowed), price$(Dollar sign illegal), char(keyword), 25th (Super script).

How to create a variable: 2 Types.

Primary Variable declaration:

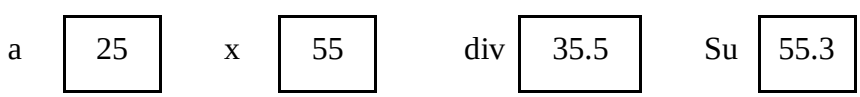
<data type> var1,var2,var3;

Ex:
```
int a,b,c;
Float div,amount;
```

Variable Initialization:

<data type> var1,var2,var3;

Ex:
```
int a=25,x=55;
Float div=35.5,su=55.3;
```



Operators and expressions:

Expressions: An expression is a sequence of Operands and Operators that reduces to single value. An expression can be simple or complex.

An operator is a syntactical token that requires an action to be taken(Ex: +,-,*,/,%)

An operand is an object on which an operation is performed. It receives an operation action. Ex: variables like a, b, c, x, y etc.

Simple expression contains only one operator. Ex a=1+6, c=N%2

A complex expression contains more than one operator. Ex 3*5+7

There are 3 category of operators:

- Unary operator
- Binary operator
- Ternary operator

Unary operator: It performs an operation on one operand.

Operator	Name	Example
+	Unary + operator	+10, +10, +a, +a,
-	Unary - operator	-10, -10, -a, -a,
*	Pointer operator	*a, *ptr
&	Address operator	&a, &b
++	Increment operator	++a/a++
--	Decrement operator	--a/a--
Here 10,a etc are single operand		

Increment operator (++): 2 Types.

- **Pre- increment: ++x/++a**

  Example-1:

  int x=7, y;

  y=++x;

  Here first the value of x is incremented to 8 and then assigned/copied to y

  x= 8 y= 8

  Example-2:

  int a=6, b;

  b= (++a) + (++a);

  Steps of evaluation:

  First the values of a is incremented to 7 and assign to b

  Next a is again incremented to 8 and assign to b: so a=8 b=15

b= 15 a= 8

- **Post- increment: x++/a++**

  Example-3:

  int x=7, y;

  y=x++;

  Here first the value of x is assigned/copied to y then x is incremented to 8

x= 8 y= 7

Example-4:

```
int a=6, b;
b= (a++) + (a++);
```

Steps of evaluation:

First the values of a is assign to b(b=6) then a is incremented to 7(a=7) a=7 is assign to b(b=6+7=13) and a is again incremented to 8 so a=8 b=13

a= 8 b= 13

Decrement operator (--): 2 Types.

- **Pre- decrement: --x/--a**

  Example-5:

  int x=7, y;

  y=--x;

  Here first the value of x is decremented to 6 and then assigned/copied to y

x= 6 y= 6

Example-6:

```
int a=6, b;
b= (--a) + (--a);
```

Steps of evaluation:

First the values of a is decremented to 5 and assign to b (b=5)Next a is again

decremented to 4 and assign to
b(b=5+4=9): so a=4 b=9

b= | 9 | a= | 4 |

Post- decrement: x--/a--

Example-7:

```
int x=7, y;
y=x--;
```

Here first the value of x is assigned/copied to y then x is decremented to 6

x= | 6 | y= | 7 |

Example-8:

```
int a=6, b;
b= (a--) + (a--);
```

Steps of evaluation:

First the values of a is assign to
b(b=6) then a is decremented to
5(a=5) a=5 is assign to
b(b=6+5=11) and a is again
decremented to 4 so a=4 b=11

a= | 4 | b= | 11 |

These are extensively used in loops(while, for, do while)

Binary operator: It performs an operation on two operands. There are 5 types.

i. Arithmetic operator
ii. Relational operator
iii. Assignment operator
iv. Logical operator
v. Bitwise operator

i.Arithmetic operator: C provides all basic arithmetic operators. Here is a brief summary of different operators

Arithmetic Operators	Name	Example
+	Addition	C=A+B
-	Subtraction	Z=X-Y
*	Multiplication	R=P*Q
/	Division (The result given is Quotient)	S=P/Q
%	Modular division (The result given is remainder) Note: modular operation is possible only on integer data types	D=A%B Here A and B should be integers

Here A,B,X,Y etc are operands.

Arithmetic expressions are of three types:

1. Integer Arithmetic
2. Real Arithmetic
3. Mixed mode Arithmetic

Integer Arithmetic: It is an expression evaluation where all the operands are of integer
data type. Let us take an example:

```
int a=10,b=5,c;
c=a/b;
printf("%d", c);
```

c will contain quotient value 2

Real Arithmetic: It is an expression evaluation where all the operands are of float data
type. Here is an example:

```
float x=2.5, y=10.0,z;
z=y/x;
printf("%f", z);
```

z will contain value 4.000000

Mixed Mode Arithmetic: In an expression if some operands are of integer type and others are of float type it is called mixed mode arithmetic expression. Here the data type of small size gets automatically converted to data type of bigger size. Here is an example for such expression:

```
int a=25;
float x=4,z;
 z=a/x;
printf("%f",z);
```

Here 'a' is of integer data type and 'x' is of float type, therefore 'a' automatically gets converted to float type (Type conversion to be discuss later) and output is: 6.25

ii.Relational operator: An operator which gives the relation between two operands. It gives either true (1) or false (0). C supports 6 relational operators.

Relational Operators	Name	Example
>	Greater Than	A >B
<	Less Than	X<Y
>=	Greater Than or Equal to	(A+B) > =(X-Y)
<=	Less Than or Equal to	(P+Q) < =(R-S)
= =	Equal to	(X+Y) = = Z
! =	Not Equal to	Z!=A

Here are some Examples on Relational operators:

Example-1:

int A=5, B=10, X=20, Y=5;
What is the truth value of expression: (A+B) = = (X-Y)?
(5+10) = = (20-5) answer is TRUE

Example-2:

int A=5, B=10, X=20, Y=10;
What is the truth value of expression: (A+B) >= (X-Y)?
(5+10)> = (20-10) answer is FALSE

iii.Assignment operator : Assignment operators are used to assign the result of an expression to a variable.

Operator	Name	Example
=	Equals(Assign)	a=b, a=10, sum=10+20
+=	Sum and Assign	a=a+b or a+=b
-=	Subtract and assign	a=a-b or a-=b
*=	Multiply and assign	a=a*b or a*=b
/=	Divide and assign	a=a/b or a/=b

%=	Modulus and assign	a=a%b or a%=b

Note: b=10; 10 assign to b

A==b; true or false.

iv.Logical operator: An operator which gives the logical relation between two operands, it gives either true(1) or false(0). C provides 3 logical operators as shown below.

Logical Operators	**Name**	**Example**
&&	Logical AND	(a<b)&&(a<c) (10<20)&&(30<10): False For true both should be true.
||	Logical OR	(a<b) || (a<c) (10<20)||(30<10): True Any one true result will be true
!	Logical NOT	!(a<b): !T Result is false Ex: !(20<10) !F Result is T

The truth table of Logical operators is as follows:

&&(Logical AND operator)			**||(Logical OR operator)**			**!(Logical NOT operator)**	
A	**B**	**A&&B**	**A**	**B**	**A||B**	**A**	**B**
0	0	0(False)	0	0	0(False)	0(False)	1(True)
0	1	0(False)	0	1	1(True)	1(True)	0(False)
1	0	0(False)	1	0	1(True)		
1	1	1(True)	1	1	1(True)		

v.Bitwise operators: C has a distinction of supporting special operator known as bitwise operators for manipulation of data at bit levels. These operators are used for testing the bits, or shifting them right or left. Bitwise operators may not be applied to float or double. Table below shows bitwise operators and their meaning.

Operators	**Meaning**	**Examples**
&	Bitwise AND	A & B
|	Bitwise OR	X | Y
>>	Bitwise Right Shift	A>>2
<<	Bitwise Left Shift	A<<2
^	Bitwise Ex-OR	**Bitwise** X ^ Y
~	Bitwise ones complement	~A

Examples for bitwise AND, OR and XOR:

A	B	A & B	A | B	A^B	~A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Example for Bitwise Right shift:

A	0	1	0	0	0	1	1	0
A>>2	0	0	0	1	0	0	0	1

Note: Meaning is shift right by 2 bits.

Example for Bitwise Left shift:

A	0	1	0	0	0	1	1	0
A<<2	0	0	0	1	1	0	0	0

Note: Meaning is shift right by 2 bits.

Ternary Operator OR Conditional Operators [?:]

An operator which performs operations on 3 operands.

Syntax:

Expression 1 ?	Expression 2 :	Expression 3 ;
(Condition)	Statement-1	Statement-2
May True or False Operand-1	Operand-2	Operand-3

Where

Expression1 is Condition
Expression2 is Statement Followed if Condition is True
Expression2 is Statement Followed if Condition is False

Meaning of Syntax:

1. Expression1 is nothing but Boolean Condition i.e it results into either TRUE or FALSE
2. If result of expression1 is TRUE then expression2 is executed
3. Expression1 is said to be TRUE if its result is NON-ZERO
4. If result of expression1 is FALSE then expression3 is executed
5. Expression1 is said to be FALSE if its result is ZERO

Example : Check whether Number is Odd or Even #include<stdio.h>
void main()
{

Where

Expression1 is Condition
Expression2 is Statement Followed if Condition is True
Expression2 is Statement Followed if Condition is False

Meaning of Syntax:

6. Expression1 is nothing but Boolean Condition i.e it results into either TRUE or FALSE
7. If result of expression1 is TRUE then expression2 is executed
8. Expression1 is said to be TRUE if its result is NON-ZERO
9. If result of expression1 is FALSE then expression3 is executed
10. Expression1 is said to be FALSE if its result is ZERO

Example : Check whether Number is Odd or EVEN

```
#include<stdio.h>
        void main()
        {
                int num;
                printf("Enter the Number : ");
                scanf("%d",&num);

(Rem = (num%2==0))?   printf("\nEven"):   printf("\nOdd");


}
```

Special operator: a) Comma Operator b) sizeof operator

Comma operator: Comma operator can be used to link the related expressions together. A comma linked list of expressions are evaluated from left to right and the value of the right most expression is the value of the combined expression For Ex.

Value=(x=10, y=5, x+y);

First 10 assign to x then 5 assign to y finally 10+5=10 assign to value. Since comma operator has the lowest precedence of all operators, parenthesis is necessary.

Sizeof Operator: The sizeof operator is a compile time operator and when used with an operand, it returns the no of bytes the operand occupies. The operand may be a variable, a constant or data type qualifier.
Ex.

```
m=sizeof(sum);
n=sizeof(long int);
```

sizeof operator also used to allocate memory space dynamically to variables during program execution.

Stings and special symbols:

Strings: Written inside the double coats.
Ex: “ABC” and “year”
Special Symbols: [] and {}

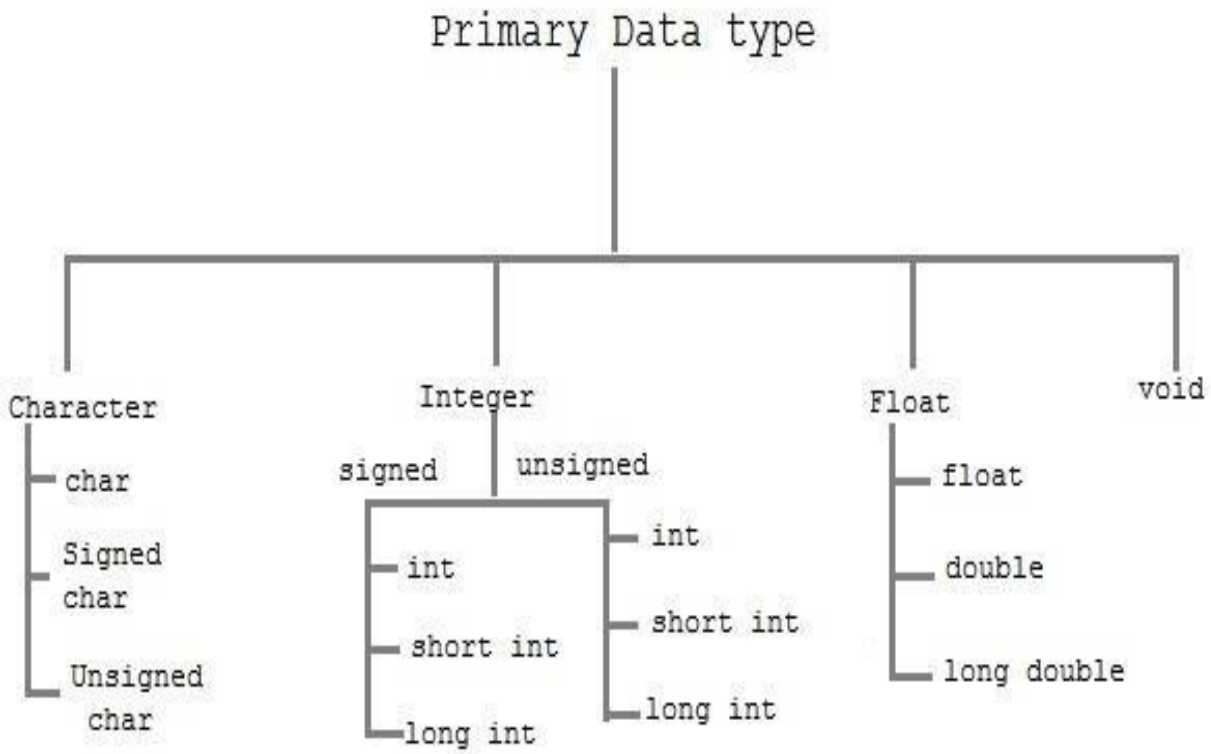
Data types in C Language

- Data types specify how we enter data into our programs and what type of data we enter. C language has some predefined set of data types to handle various kinds of data that we use in our program. These datatype have different storage capacities.
- C language supports 3 different type of data types:

Primary/ Primitive data types/Fundamental data types These are fundamental data types in C namely integer (int), floating (float), character (char) and void.

Derived data types: Derived data types are like arrays, functions, structures and pointers.

User defined data types:



Integer: An integer is a whole number (a number without a fractional part). It can be positive or negative numbers like 1, -2, 3, etc., or zero.

It is called signed integer if it is having both +ve and –ve values and unsigned integer if it is having only +ve values .

Range of unsigned number **in n bit** machine is **0 to 2^n-1**

Range of signed number **in n bit** machine is **-2^{n-1} to $+2^{n-1}$-1**

No. Of bits in machine	Type	Size	Range of unsigned int	Range of signed int
16 bit machine	Short int (short)	2 bytes	0 to 2^{16}-1 (0 to 65,535)	-2^{15} to $+2^{15}$-1 (-32768 to 32767)
	int	2 bytes	0 to 2^{16}-1 (0 to 65,535)	-2^{15} to $+2^{15}$-1 (-32768 to 32767)
	long int	4 bytes	0 to 2^{31}-1 (0 to 4294967295)	-2^{31} to $+2^{31}$-1 (-2147483648 to 2147483647)
32 bit machine	int	4 bytes	0 to 2^{31}-1 (0 to 4294967295)	-2^{31} to $+2^{31}$-1 (-2147483648 to 2147483647)
	long int	4 bytes	0 to 2^{31}-1 (0 to 4294967295)	-2^{31} to $+2^{31}$-1 (-2147483648 to 2147483647)

float: Since int cannot store higher range of numbers and numbers with fractional part cannot be represented So, the float is a keyword used to define floating point numbers in C. Floating point numbers have 6 decimal digits precision after decimal point i.e. it will show only 6 digits after decimal point. For example if the number is 67.78986458738 then it will show 67.789864

n bit machine	Size of float	Range of float
16 bit machine	4 bytes	3.4E-38 to 3.4E38
32 bit machine	8 bytes	1.7E-308 to 1.7E308

double: A double is a keyword used to define long floating point numbers in C. It is considered as floating-point numbers but the range will be more. Double have 12 decimal digits precision after decimal point i.e. it will show 12 digits after decimal point. For example if the number is 67.78986458738 then it will show 67.789864587380

n bit machine	Size of float	Range of float
16 bit machine	8 bytes	1.7E-308 to 1.7E308
32 bit machine	16 bytes	3.4E-4932 to 1.1E4932

char: A char keyword is used to define a single character.

n bit machine	Size of float	Range of unsigned char	Range of signed char
16/32 bit machine	1 byte	0 to 2^8-1 (0 to 255)	-2^7 to $+2^7$-1 (-128 to 127)

void: A void is an empty data type. No value is associated with void so no memory will be allocated to void.

n bit machine	Size of float	Range of float
16 /32 bit machine	0	No value

The range of integer data type is given in **limits.h** header file and range and precision of float data type is given in **float.h** header file. We can use **sizeof()** operator to find out the size of any data type in any system.

Conversion Specification (Format Specifiers): The function scanf() and printf() make use of conversion specifications to specify the type and size of data. Each specifier must begin with % sign. Some are given below.

Conversion Specifications for scanf()

Conversion Character(Format Specifier)	Meaning
%c	Read a single character
%d	Read a signed decimal integer
%i	Read all signed integers (decimal, octal, hexadecimal). Decided by prefix of integer.
%u	Read an unsigned decimal integer
%o	Read an unsigned octal integer
%x or %X	Read an unsigned hexadecimal integer
%f or %e	Read a floating point number
%s	Read a string

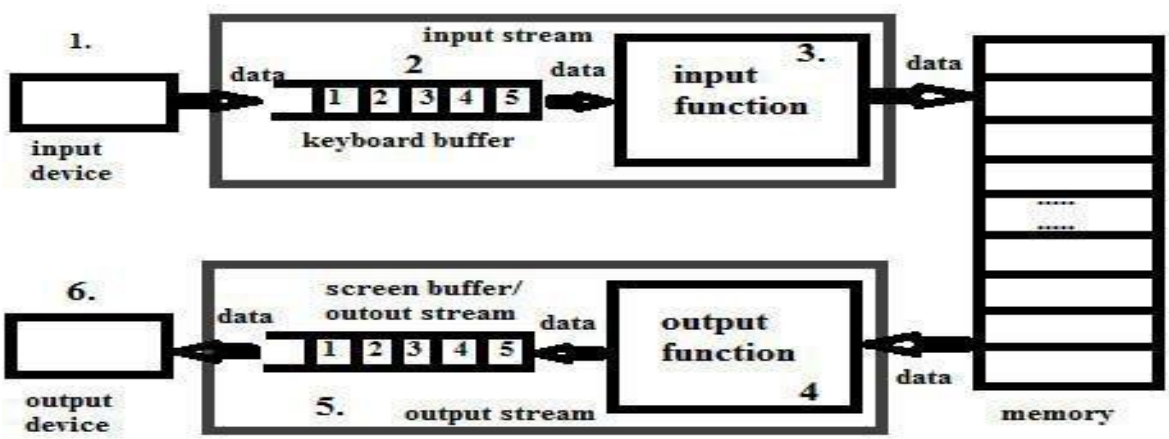
Conversion Specifications for printf()

Conversion Character(Format Specifier)	Meaning
%c	Prints a single character
%d or %i	Prints a signed decimal integer
%h	Prints a short integer
%ld	Prints a long integer
%u	Prints an unsigned decimal integer
%o	Prints an unsigned octal integer
%x	Prints an unsigned hexadecimal integer using a,b,c,d,e,f
%X	Prints an unsigned hexadecimal integer using A,B,C,D,E.F
%f	Prints a floating point number(by default 6digits Printed after the point)
%e	Prints a floating point number in exponential form. For ex: 2.8e30
%lf	Prints a number of type double
%Lf	Prints a number of type long double
%s	Prints a string
%p	Prints an address in hexadecimal format.

Input and Output Functions

1. Data is entered from keyboard.
2. The data is stored in temporary memory called keyboard buffer. It holds characters until we press enter.
3. Keyboard function converts data into appropriate data type and stored in memory locations using variables.
4. Using output function, it is read from memory and stored in temporary storage area called screen buffer.
5. The data is then visible on the screen.

The above steps are illustrated in following Figure.



Types of input output functions:

1.Formatted Input Function: scanf() is the library function defined in **stdio.h** header file which can be used to enter the data in formatted way. This function takes all types of values (numeric, character, string) as input. The scanf() can be written as:

scanf("Format String",address1,address2);

i. This function should have at least two parameters. First is format string, which contains conversion specification characters and should be written in double quote. The conversion specification characters may be one or more depending upon the number of inputs you want to read.

ii. The other parameter is called as address list, where variable name should be written with **&** sign as prefix. Here **&** sign will represent the address of the variable in memory and it is known as **address operator.**

Important: For **string** type variable **&** sign in not required**.**

For example:

```
#include<stdio.h>
main()
{
        int marks;
        scanf("%d", &marks);
}
```

```
#include<stdio.h>
main()
{
        char ch;
        scanf("%c", &ch);
}
```

```
#include<stdio.h>
main()
{
        float height;
        scanf("%f", &height);
}
```

```
#include<stdio.h>
main()
{
        int marks;
        float height;
        char ch;
        scanf("%d %f %c", &marks, &height, &ch);
}
```

2.Formatted Output Function: printf() is a library function defined in **stdio.h** header file which will take data stored at given location and prints it on monitor. This function prints all types of values (numeric, character, string) as input.

The printf() can be written as:

printf("Format String", variable1,variable2,.....);

or

printf("write the message to be printed");

ii. This function can be used to print a message by enclosing it in "" double quote. Here conversion specification is not required. For ex second syntax.

ii. This function also can be used to print the value of one or more variables. Here, at least two parameters are required. First is format string, which contains conversion specification characters and should be written in double quote. The conversion specification characters may be one or more depending upon the number of values you want to print.

iii.Here **&** sign (address operator) is not required for any data type.

iv. The number of conversion specification characters and number of variables in address list must be same. We can also include extra message within double quote with specifier.

For example:

```
#include<stdio.h>
   Void main()
{
        int marks;
        printf("%d", marks);

}
```

```
#include<stdio.h>
   void  main()
{
        int marks;
        printf("Marks = %d", marks);

}
```

v.We can format our output according to our way**. For example ,** Suppose we want to print date in format (10-02-2016) so, we can write as

```
#include<stdio.h>
main()
{
        int day, month, year;
        printf("Date: %d-%d-%d",day, month, year);
}
```
Output: Date: 10-02-2016

vi. If you want to print only 'n' digits after decimal point in floating point number, we can use **%.nf.** Here 'n varies according to the number of digits to be printed after decimal point in output. **For example,** Suppose your float variable is having value as 78.5678295 and you want to print only

78.567 then:

```
#include<stdio.h>
main()
{
        float marks=78.5678295;
        printf("Marks = %.3f,marks);
}
```
Output: Marks: 78.567

Some special formatting of scanf() and printf() functions:

Format for Integer input:

%wd: here **d** is the conversion specification character (format specifier) for integer and **w** is an integer number specifying the maximum field width (number of digits) of input data.

For Example, if:

scanf("%2d%3d", &a, &b);

i. If length of input is less than or equal to the given field width w, the input values are unaltered and stored in given variables. For example, if input is 6 394 then 6 will be stored in a and 394 will be stored in b.

ii. If length of input is more than the given field width w, the input values are altered and stored in given variables. For example, if input is 269 3945 then 26 will be stored in **a** and 9 will be stored in **b.** Here 9 is just after 6 so scanning for variable **b** will start from here and after that space is there so it will terminate.

Format for Integer output:

%wd: here **d** is the conversion specification character (format specifier) for integer and **w** is an integer number specifying the maximum field width (number of blocks allocated) of output data. For Example, if:

printf("%2d%3d",a,b);

i. If length of variable is less than the given field width w, the output values will be printed in right justified manner. For example, if a=6 and b=39 then it will be printed as

	6

	3	9

ii. If length of variable is equal to the given field width w, the output values will be occupying all blocks. **For example if a=63 and b=399 then it will be printed as**

6	3

3	9	9

iii.If length of variable is more than the given field width w, extra blocks will be given and output values will be printed as it is. **For example, if input is 269 3945**

❖

2	6	9

3	9	4	5

Format for floating point input:

%wf: here **f** is the conversion specification character (format specifier) for float number and **w** is an integer number specifying the maximum field width (number of digits including decimal point, before decimal and after decimal) of input data.

scanf("%3f%4f",&a,&b);

i.If length of input is less than or equal to the given field width w, the input values are unaltered and stored in given variables. For example if input is 6 3.94 then 6.0 will be stored in a and 3.94 will be stored in b.

ii. if length of input is more than the given field width w, the input values are altered and stored in given variables. For example if input is 2.69 39.45 then 2.6 will be stored in a and 9.00 will be stored in b. Here 9 is just after 6 so scanning for variable b will start from here and after that space is there so it will terminate**.**

Format for floating point output:

%w.nf: here **f** is the conversion specification character (format specifier) for float number and **w** is an integer number specifying the maximum field width (number of digits including decimal point, before decimal and after decimal) and **n** is the number of digits printed after decimal point.

printf("%4.1f%7.2f",a,b);

i. If length of variable is less than the given field width w, the output values will be printed in right justified manner. If the number of digits after decimal is more than n then digits are rounded off. **For example, if a=6 and b=39 then it will be printed as**

	6	.	0

		3	9	.	0	0

ii. If length of variable is equal to the given field width w, the output values will be occupying all blocks. **For example if a=63.5 and b=3567.88 then it will be printed as**

6	3	.	5

3	5	6	7	.	8	8

iii. If length of variable is more than the given field width w, extra blocks will be given to the digits before decimal and after decimal it will follow the same concept as above.

For example if input is 269.689 69.576799

2	6	**9**	.	7	

		6	**9**	.	5	8	

here, for the width for **a** is 4 and decimal digits after decimal is 1 and the value of a that

is 269.689 is having total width as 7 so left part of decimal point will be printed as it is and only 1 rounded off digit will be printed after decimal

Error Types :

While writing c programs, errors (bugs) may occur unwillingly which may prevent the program to compile and run correctly as per the expectation of the programmer. Basically there are three types of errors in c programming:

1. Syntax Errors
2. Logical Errors
3. RuntimeErrors

Syntax Errors

When the rules of the c programming language are not followed, the compiler will show syntax errors. For example, consider the statement,

int a,b:

The above statement will produce syntax error as the statement is terminated with : rather than ;

Logical Errors

Logical errors are the errors in the output of the program. The presence of logical errors leads to undesired or incorrect output and are caused due to error in the logic applied in the program to produce the desired output.

Also, logical errors could not be detected by the compiler, and thus, programmers have to check the entire coding of a c program line by line.

Runtime Errors

C runtime errors are those errors that occur during the execution of a c program and generally occur due to some illegal operation performed in the program.

Examples of some illegal operations that may produce runtime errors are:

- Dividing a number by zero
- Trying to open a file which is not created
- Lack of free memory space

It should be noted that occurrence of these errors stops the program execution.

Precedence and Associativity of operators:

Consider the expression: 2+3*4

Here we have two operators addition and multiplication. If addition is performed before multiplication the result will be 20 and if multiplication is performed first the result will be 14. So, to remove this ambiguity (having two different results) in result while evaluation of expression having more than one operators, there are certain precedence of priority and associativity rules defines in C which must be followed

Precedence (Priority): If operators present in the expression are having different precedence then we have to solve the higher precedence operator first. For example:

C= 2+3*(15-6)

Here according to precedence bracket will be solved first and then multiplication then addition and at last assignment.

Order of priority: () next [] next {} next * / % next + - next && next ||

C=2+3*(15-6)
C=2+3*9
C=2+27
C=29

Associativity: If in all the operators present in the expression some are having equal precedence than it is confusing that which one should be evaluated first among them. So, here in this case we have to see the associativity for equal priority operators and if it is left to right then those will be evaluated in left to right sequence and if the associativity is right to left than those will be evaluated in right to left sequence. For example:

Precedence and Associativity of operators:

Operator	Description	Associativity
( )	Parentheses (function call)	left-to-right
[ ]	Brackets (array subscript)	
.	Member selection via object name	
->	Member selection via pointer	
++ --	Postfix increment/decrement	
++ --	Prefix increment/decrement	right-to-left
+ -	Unary plus/minus	
! ~	Logical negation/bitwise complement	
(*type*)	Cast (convert value to temporary value of *type*)	
*	Dereference (used in pointers)	
&	Address (of operand)	
sizeof	Determine size in bytes on this implementation	
* / %	Multiplication/division/modulus	left-to-right
+ -	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right

< <= > >=	Relational less than/less than or equal to Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR (XOR)	left-to-right
|	Bitwise inclusive OR (OR)	left-to-right
&&	Logical AND	left-to-right
||	Logical OR	left-to-right
? :	Ternary conditional	right-to-left
= += -= *= /= %= &= ^= |= <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left

,	Comma (separate expressions)	left-to-right

Consider

C= 2+3*6+7-4%2

Here total 5 operators are there in which according to precedence multiplication and modulus are having equal precedence among them and addition and subtraction are having equal precedence among them but multiplication and modulus have higher precedence than addition and subtraction. So, here we have to see the

associativity for multiplication and modulus which is left to right and also for addition and subtraction it is left to right. So, Final evaluation sequence is:

C= 2+**3*6**+7-4%2 * and % Having equal priority but associativity left to right.
C= 2+18+7-**4%2**
C= 2+18+7-0 + and - Having equal priority but associativity left to right.
C= 2+18+7-**4%2**
C= 20+7-0
C=27-0
C=27

Some Expression Evaluation: Here the operator evaluation sequence is written in bold per step.

1. a=8, b=4,c=2, d=1, e=5, f=20 then valuate the given expression.
   a +b - (c + d) *3 %e + f/9
   8+4-**(2+1) ***3%5 + 20/9 [bracket highest priority]
   8+4-**3*3**%5+20/9 [* % and / are of equal priority but left to right associativity]

8+4-**9 %5**+20/9
8+4-4+**20/9**
8+4-4+2 [+ - + are of equal priority but left to right associativity]
12-4+2
8+2
10

2. a=8, b=3, c=2, d=3, e=2, f=11 then evaluate the following expression
a-b || (a-b*c)+d&&e-f%3 [bold part of expression shows the priority and associativity in each step]
8-3 || (8-**3*2**) +3 && 2-11%3
8-3 || **(8-6)** +3 && 2-11%3
8-3 || 2 + 3&&2-**11%3**
8-3 || 2+3 && 2-2
5 || **2+3** && 2-2
5 || 5 && **2-2**
5 || **5 && 0** [Non Zero i.e. 5 here is considered as true i.e. 1, && higher Priority than ||]
5 || 0
1

3. a=2, b=3, c=4 then evaluate the given expression

x= a<=b || b==c
x= **2<=3** || 3==4 [bold part of expression shows the priority and associativity in each step]
x= 1 || **3==4**
x= **1 || 0**
x=1

4. v=17, w=5, x=6, y=3, z=5 then evaluate the following expression.
v% 6-w/2+(x*y-5)/z
17%6-5/2+(**6*3**-5)/5 [bold part of expression shows the priority and associativity]
17%6-5/2+**(18-5)**/5
17%6-5/2+13/5
5-**5/2**+13/5
5-2+**13/5**
5-2+2
3+2
5

5 . Find the output:

```
#include<stdio.h>
    Main ()
```

```
{
   int i= -3, j=2, k=0, m;
   m = ++i && ++j
   && ++k;
   printf("%d, %d,
   %d, %d\n", i, j, k,
   m);
}
```

Output: -2, 3, 1, 1

Explanation:

Step 1: int i=-3, j=2, k=0, m; here variable i, j, k, m are declared as an integer type and variable i, j, k are initialized to -3, 2, 0 respectively.
Step 2: m = ++i && ++j && ++k;
becomes m = -2 && 3 && 1; because all variable are pre-incremented.
Becomes m = As non - zero values are 1 so, m=1 && 1 && 1; Hence this statement becomes TRUE. So it returns '1'(one). Hence m=1.
Step 3: printf("%d, %d, %d, %d\n", i, j, k, m); In the previous step the value of i,j,k are incremented by '1'(one).
Hence the output is "-2, 3, 1, 1".

6.Find the output:

```
#include<stdio.h>
Main ()
{
   int x=4, y, z;
   y = --x;
   z = x--;
   printf("%d, %d, %d\n", x, y, z);
 }
```

Output: 2, 3, 3

Explanation:

Step 1: int x=4, y, z; here variable x, y, z are declared as an integer type and variable x is initialized to 4.
Step 2: y = --x; becomes y = 3; because (--x) is pre-decrement operator.
Step 3: z = x--; becomes z = 3;. In the next step variable x becomes 2, because (x--) is post-decrement operator.
Step 4: printf("%d, %d, %d\n", x, y, z); Hence it prints "2, 3, 3".

7.Find the output:

```
#include<stdio.h>
int main ()
{
```

```
int a=100, b=200, c;
c = (a == 100 || b > 200);
printf("c=%d\n", c);
return 0;
}
```

Output: c=1

C Type Conversion:

When variables and constants of different types are combined in an expression then they are both converted to same data type. The process of converting one predefined type into another is called type conversion.

Type conversion in c can be classified into the following two types:

1. **Implicit Type Conversion**
   - When the type conversion is performed automatically by the compiler without programmer's intervention, such type of conversion is known as **implicit type conversion** or **type promotion.**
   - The implicit type conversion is done in 2 ways by compiler:

     In Arithmetic expression: The compiler converts all operands of other data types into the data type of the largest operand by following this hierarchy:

Char or short int →int→unsigned int→long int→→float→double→long double

The sequence of rules that are applied while evaluating expressions are given below: All short and char are automatically converted to int, then,

1. If either of the operand is of type long double, then all others will be converted to long double and result will be long double.
2. Else, if either of the operand is double, then all others are converted to double.
3. Else, if either of the operand is float, then all others are converted to float.
4. Else, if either operand is long int then all others will be converted to long int. Etc.

For Example:

```
#include<stdio.h>
   main ()
{
      int num1=7, num2=5;
      float num3=8.7,sum;
       sum= num1+num2+num3;
      printf("Sum=%f",sum);
```

```
}
```

Output:
Sum= 20.700000

Explanation: In the line sum=num1+num2+num3 here num1 and num2 will be converted into float because num3 is in float see the hierarchy and point 3 above.

In Assignment expression: The compiler converts all operands of **right hand side** data types into the data type of the **left side** operand. For example:

```
#include<stdio.h>
main()
{
        int a=54, d;
       char b='A', e;
       float c=68.76, f;
       d=b;
       e=a;
       f=a;
       printf("a=%d, b=%c, c=%f, d=%d, e=%c, f=%a", a, b, c, d, e, f);
}
```

Output:
a=54, b=A, c=68.760000, d=65, e=6,f=54.000000

Explanation: In the line d=b 'A' will be converted into integer (ASCII value) i.e. 65.
In the line e=a 54 will be converted into char (ASCII character) i.e. 6.
In the line f=a 54 will be converted into float i.e. 54.000000

2. Explicit Type Conversion

The type conversion performed by the programmer by posing the data type of the expression of specific type is known as explicit type conversion.
The explicit type conversion is also known as **type casting**. Type casting in c is done in the following form:
(data_type)expression;

where, *data_type* is any valid c data type, and *expression* may be constant, variable or expression.

```
#include<stdio.h>
void main()
{
 int num1=7, num2=5;
        float divison;
        division= (float)num1/num2;
        printf("Division=%f",division);
}
```

Output:

Sum= 1.400000

Explanation: In the statement division= (float)num1/num2; **either of num1 and num2 or both should be converted into float because in C integer divided by integer gives only integer part of division for example 7/5 will give only 1 if no type casting is used.**

The following rules must be followed while converting the expression from one type to another to avoid the loss of information:

1. All integer types to be converted to float.
2. All float types to be converted to double.
3. All character types to be converted to integer.